beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in beamer documents. Named overlay specifications are very handy both during edition and to manage complex and variable beamer overlay specifications. In particular, they allow to replace raw numbers in beamer <...> overlay specifications by logical identifiers. Demonstration files are available for download as part of the development repository. As a side effect, this is another solution to this latex.org forum query.

Contents

1 Implementation

1.1 Package declarations

```
\NeedsTeXFormat{LaTeX2e}[2025/01/01]
\ProvidesExplPackage
  {beanoves}
  {2025/06/03}
  {1.0}
  {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into beamer specifications based on this data model.

The development is made easier due to a facility layer over expl.

GOBBLED

```
\regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
  \A \s* (?:
```

- 2 → V

```
    ( [^:]+? )
```

- 3, 4, 5 → A : Z? or A :: L?

```
    | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
```

- 6, 7 → ::(L:Z)?

```
    | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
```

- 8, 9 → :(Z::L)?

```
    | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
  )
  \s* \Z
}
```

GOBBLED

1.3 Main scanner

GOBBLED

1.3.1 Base grammar

GOBBLED

1.3.2 Storage

GOBBLED

1.3.3 AST exportation policy

This is used by \BNVS_end_scan: to wrap the AST created during the scanning process before it is exported. The case of an empty created AST is sometimes special.

\BNVS_ast_prefix_set:n \BNVS_ast_prefix_set:n {⟨prefix:w⟩}

Set the function \BNVS_ast_prefix:w to expand to \exp_not:n { ⟨prefix:w⟩}. This is used to define \BNVS_ast_plc:n, when the arguments must be scanned prior to knowing which function ⟨prefix:w⟩ fits.

```
\cs_new:Npn \BNVS_ast_prefix:w {}

\cs_new:Npn \BNVS_ast_prefix_set:n #1 {
  \cs_set:Npn \BNVS_ast_prefix:w { \exp_not:n { #1 } }
}
```

\BNVS_ast_plc_wrap:n \BNVS_ast_plc_wrap:n {⟨template:n⟩}
\BNVS_ast_plc_wrap_pfx:n \BNVS_ast_plc_wrap_pfx:n {⟨template:n⟩}
\BNVS_ast_plc_wrap_mpt:n \BNVS_ast_plc_wrap_grp:n {⟨template:n⟩}
\BNVS_ast_plc_wrap_grp:n \BNVS_ast_plc_wrap_mpt:n {⟨template:n⟩}

Set the AST exportation policy through the meaning of the function \BNVS_ast_plc:n. The ⟨template:n⟩ is by default ##1. With the first variant, nothing is exported if the input AST is blank. With the pfx variant, only the ⟨prefix:w⟩ is exported when the input AST is blank. The mpt and grp variants never export an empty AST. The first one exports the input AST as is whereas the second one exports the input AST between a pair of braces.

```
\cs_new:Npn \BNVS_ast_plc_wrap:n #1 {
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \tl_if_blank:nF { ##1 } {
      \BNVS_ast_prefix:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
      \exp_not:n { #1 }
    }
  }
}
\BNVS_ast_plc_wrap:n { #1 }

\cs_new:Npn \BNVS_ast_plc_wrap_pfx:n #1 {
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \BNVS_ast_prefix:w
    \tl_if_blank:nF { ##1 } {
```

and \exp_not:n{⟨xprt:n⟩}.

```
      \exp_not:n { #1 }
    }
  }
}

\cs_new:Npn \BNVS_ast_plc_wrap_mpt:n #1 {
s
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \BNVS_ast_prefix:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
    \exp_not:n { #1 }
  }
}

\cs_new:Npn \BNVS_ast_plc_wrap_grp:n #1 {
s
  \cs_set:Npn \BNVS_ast_plc:n ##1 {
    \BNVS_ast_prefix:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
    \exp_not:n { { #1 } }
  }
}
```

1.3.4 Branch functions

Alter branch The alter branch is used when the last character scanned is a delimiter, and there is some character available for scanning but it does not fit the current requirement as defined by the scan function called.

\BNVS_scanned_alter:w — \BNVS_scanned_alter:w ⟨input stream⟩
\BNVS_scanned_alter_set:n — \BNVS_scanned_alter_set:n {⟨alter:w⟩}

The \BNVS_scanned_alter:w branch function is called by \BNVS_scan_alter:Fw once ⟨alter⟩ has been scanned from the head of the ⟨input stream⟩ (greedy match). \BNVS_scanned_alter_set:n sets the meaning of \BNVS_scanned_alter:w to ⟨alter:w⟩.

\BNVS_scan_alter:Fw — \BNVS_scan_alter:Fw {⟨else:w⟩} ⟨input stream⟩
\BNVS_scan_alter:TFw — \BNVS_scan_alter:TFw {⟨yes:w⟩} {⟨else:w⟩} ⟨input stream⟩
\BNVS_scan_alter_wrap:n — \BNVS_scan_alter_wrap:n {⟨alter:TFw⟩}

Once ⟨alter⟩ has been scanned from the head of the ⟨input stream⟩ (greedy), calls \BNVS_scanned_alter:w branch function, otherwise execute ⟨else:w⟩ instructions. Scanning ⟨alter⟩ is driven by ⟨alter:TFw⟩.

```
\cs_new:Npn \BNVS_scan_alter:Fw #1 {
  \BNVS_scan_alter:TFw {
    \BNVS_scanned_alter:w
  } {
    #1
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}

\cs_new:Npn \BNVS_scan_alter:TFw #1 #2 {
```

At the top level, the ⟨else:w⟩ instructions are always executed.

```
  #2
}

\cs_new:Npn \BNVS_scan_alter_wrap:n #1 {
  \cs_set:Npn \BNVS_scan_alter:TFw ##1 ##2 {
```

```
    #1
  }
}
\BNVS_scan_alter_wrap:n { #2 }
```

Colon branch

`\c__bnvs_colons_regex` For ranges defined by a colon syntax. One catching group for more than one colon.

```
\regex_const:Nn \c__bnvs_colons_regex { \s* :(:*) \s* }
```

(*End of definition for* `\c__bnvs_colons_regex`.)

`\BNVS_scan_colon:Fw`	`\BNVS_scan_colon:Fw {`⟨*else:*⟩`}` ⟨*input stream*⟩
`\BNVS_scanned_one_colon:w`	`\BNVS_scan_colon:Fw {`⟨*else:w*⟩`}` ⟨*input stream*⟩
`\BNVS_scanned_one_colon_set:n`	`\BNVS_scan_one_colon:w` ⟨*input stream*⟩
`\BNVS_scanned_colons:w`	`\BNVS_scan_one_colon_set:n {`⟨*one_colon:w*⟩`}` `\BNVS_scan_colons:w` ⟨*input stream*⟩
`\BNVS_scanned_colons_set:n`	`\BNVS_scan_colons_set:n {`⟨*colons:w*⟩`}`

On scanning a standalone colon from the ⟨*input stream*⟩, calls the branch function `\BNVS_scanned_one_colon:w`. On scanning multiple colons from the ⟨*input stream*⟩, calls the `\BNVS_scanned_colons:w` branch function. In other cases, execute the ⟨*else:w*⟩ instructions.

One shot utility function only used by `\BNVS_scan_colon:Fw`

```
\cs_new:Npn \BNVS_scanned_colons_regex:n #1 {
  \tl_if_empty:nTF { #1 } {
    \BNVS_scanned_one_colon:w
  } {
    \BNVS_scanned_colons:w
  }
}
```

`\peek_regex_remove:NnTF` is buggy in L3 programming layer <2025-01-18>.

```
\cs_new:Npn \BNVS_scan_colon:Fw #1 {
  \peek_regex_replace_once:NnTF \c__bnvs_colons_regex { \cB{ \1 \cE} } {
    \BNVS_scanned_colons_regex:n
  } {
    #1
  }
}
```

Comma branch

`\c__bnvs_comma_regex` Comma as a separator.

```
\regex_const:Nn \c__bnvs_comma_regex { \s* , \s* }
```

(*End of definition for* `\c__bnvs_comma_regex`.)

`\BNVS_scan_comma:Fw` `\BNVS_scan_comma:Fw {⟨else:w⟩} ⟨input stream⟩`
`\BNVS_scanned_comma:w` `\BNVS_scan_comma:w ⟨input stream⟩`
`\BNVS_scanned_comma_set:n` `\BNVS_scan_comma_set:n {⟨comma:w⟩}`

Once a comma has just been scanned from the head of the ⟨*input stream*⟩, calls `\BNVS_scanned_comma:w` branch function, otherwise execute ⟨*else:w*⟩ instructions. `\BNVS_scan_comma_set:n` sets the meaning of `\BNVS_scan_comma:w` to ⟨*comma:w*⟩.

```
\cs_new:Npn \BNVS_scan_comma:Fw #1 {
```

`\peek_regex_remove:NnTF` is buggy in L3 programming layer <2025-01-18>.

```
  \peek_regex_replace_once:NnTF \c__bnvs_comma_regex {} {
    \BNVS_scanned_comma:w
  } {
    #1
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}
```

Close branch The scanning process only applies to characters of category codes space and other. It definitely stops at the first token that is not of that category. When encontering some particular character combinations, various branch functions are called.

`\c__bnvs_close_regex` Closing delimiters.

```
\regex_const:Nn \c__bnvs_close_regex { \s* (%([{
  \}|\]|\)
  ) \s*
}
```

(*End of definition for* `\c__bnvs_close_regex`.)

`\BNVS_scan_close:Fw` `\BNVS_scan_close:Fw {⟨else:⟩} ⟨input stream⟩`
`\BNVS_scanned_close:nw` `\BNVS_scanned_close:nw ⟨clositer⟩ ⟨input stream⟩`
`\BNVS_scanned_close_set:` `\BNVS_scanned_close:w ⟨input stream⟩`
`\BNVS_scanned_close:w` `\BNVS_scanned_close_set:n ⟨close:w⟩`

When a closing delimiter is scanned, call `\BNVS_scanned_close:nw`, which in turn calls `\BNVS_scanned_close:w` with that delimiter as argument. Otherwise execute ⟨*else:*⟩ code.

`\BNVS_scanned_close:w` is a branch function called when the closing delimiter ⟨*clositer*⟩ has just been scanned. After it manages eventual errors in balancing delimiters, it calls the branch function `\BNVS_scanned_close:w` used many times. When redefined (for example by `\__bnvs_scan_delimited:NNnFw`), the `\BNVS_scanned_close:w` function does not forward to the eponym function, instead it must know where to go next.

```
\cs_new:Npn \BNVS_scan_close:Fw #1 {
  \peek_regex_replace_once:NnTF \c__bnvs_close_regex { \cB{ \1 \cE} } {
```

```
    \BNVS_scanned_close:nw
  } {
    #1
  }
}

\cs_new:Npn \BNVS_scanned_close:nw #1 {
  \BNVS_error:n { Unexpected~delimiter~`#1' }
  \BNVS_scanned_close:w
}

\tl_map_inline:nn {{close}{comma}{one_colon}{colons}{alter}} {
  \cs_new:cpn { BNVS_scanned_#1_set:n } ##1 {
    \cs_set:cpn { BNVS_scanned_#1:w } {
      ##1
    }
  }
```

Do nothing operation at the top level.

```
  \cs_new:cpn { BNVS_scanned_#1:w } {
  }
}
```

Stop branch GOBBLED

1.3.5 Begin and end of scanning processes

GOBBLED

1.3.6 Main scan functions

GOBBLED

1.3.7 ⟨*Rnd*⟩, ⟨*Sqr*⟩ and ⟨*Crl*⟩ units

We assume that delimiters are paired and balanced. Nevertheless, some errors are catched and properly managed. Here is a grammar for material enclosed between delimiters

```
⟨Rnd(⟨blnc⟩)⟩ ::= ⟨Rnd_open⟩ ⟨blnc⟩ ⟨Rnd_close⟩
⟨Rnd_open⟩    ::= ⟨spc⟩ '(' ⟨spc⟩
⟨Rnd_close⟩   ::= ⟨spc⟩ ')' ⟨spc⟩
⟨Sqr(⟨blnc⟩)⟩ ::= ⟨Sqr_open⟩ ⟨blnc⟩ ⟨Sqr_close⟩
⟨Sqr_open⟩    ::= ⟨spc⟩ '[' ⟨spc⟩
⟨Sqr_close⟩   ::= ⟨spc⟩ ']' ⟨spc⟩
⟨Crl(⟨blnc⟩)⟩ ::= ⟨Crl_open⟩ ⟨blnc⟩ ⟨Crl_close⟩
⟨Crl_open⟩    ::= ⟨spc⟩ '{' ⟨spc⟩
⟨Crl_close⟩   ::= ⟨spc⟩ '}' ⟨spc⟩
```

with different variations of associate AST's. (missing description for ⟨*Sqr*⟩ and ⟨*Crl*⟩ are similar)

⟨*Rnd(⟨blnc⟩)*⟩* ::= ⟨*prefix:w*⟩ { ⟨*blnc*⟩* }
| { ⟨*blnc*⟩* }
| ⟨*prefix:w*⟩ ⟨*blnc*⟩*
| ⟨*blnc*⟩*

The exported AST also depends on the exportation policy.

\BNVS_scan_close:nTTNnw
\BNVS_scan_close:nnTTNnw

\BNVS_scan_close:nTTNnw {⟨*preamble:*⟩} {⟨*close:*⟩} {⟨*stop:*⟩} ⟨*close*⟩ {⟨*scan:w*⟩}
\BNVS_scan_close:nnTTNnw {⟨*xprt:n*⟩} {⟨*prefix:w*⟩} {⟨*close:*⟩} {⟨*stop:*⟩} ⟨*close*⟩ {⟨*scan:w*⟩}

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using ⟨*scan:w*⟩. Implementation detail: we begin a scanning group that will be closed on scanning the ⟨*close*⟩ delimiter, or a different closing delimiter, allways at the same level. The ⟨*scan:w*⟩ function scans the ⟨*input stream*⟩, and is expected to end on a closing delimiter at the current level by sending \BNVS_scanned_close:w, which meaning is ⟨*close:*⟩ followed by \BNVS_scanned_alter:w. This is the normal instruction flow.

```
\cs_new:Npn \BNVS_scan_close:nTTNnw #1 #2 #3 #4 #5 {
  \BNVS_scan:nnnnnnnnw {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \cs_set:Npn \BNVS_scanned_close:nw ##1 {
```

Anyways, branch here.

```
      \BNVS_scanned_close:w
    }
    #1
  } {
```

Any branch function but the two last raises and forwards to \BNVS_scanned_close:w,

```
    \BNVS_error:n { Internal~error(alter:w) }
    \BNVS_scanned_close:w
  } {
    \BNVS_error:n { Internal~error(one_colon:w) }
    \BNVS_scanned_close:w
  } {
    \BNVS_error:n { Internal~error(colons:w) }
    \BNVS_scanned_close:w
  } {
    \BNVS_error:n { Internal~error(comma:w) }
    \BNVS_scanned_close:w
  } {
    \BNVS_end_scan_no_I:
    #2
    \BNVS_scanned_alter:w
```

Close branch. End the group and execute ⟨*close:w*⟩ instructions. Intermediate groups should be closed appropriately with the right definition of branch functions. Clearing the `I` variable before returning has the side effect not to export the actual value of `I` and keep what is already in the containing group.

```
  } {
```

or the ⟨*stop:*⟩ instructions after the group ends.

```
      \BNVS_error:n { Missing~`#4'. }
      \BNVS_end_scan:
      #3
    } {
```

One group level for the contents.

```
      #5
    }
}
```

GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED GOBBLED

`\c__bnvs_Crl_open_regex` Curly opening delimiter.

(*End of definition for* `\c__bnvs_Crl_open_regex`.)

```
\regex_const:Nn \c__bnvs_Crl_open_regex { \s* \{ %}
  \s* }
```

`\BNVS_scan_Crl:nTTnFw` `\BNVS_scan_Crl:nTTnFw {⟨preamble:⟩} {⟨close:⟩} {⟨stop:⟩} {⟨scan:w⟩} {⟨no:w⟩} ⟨input stream⟩`

Scan ⟨*Crl*⟩ unit with the help of ⟨*scan:w*⟩, then execute one of `\BNVS_scanned_close:` or `\BNVS_scanned_stop:` instructions. ⟨*scan:w*⟩ must end with one of the branch functions `\BNVS_scanned_close:w` or `\BNVS_scanned_stop:`. Otherwise execute ⟨*no:w*⟩ instructions.

```
\group_begin:
\char_set_catcode_group_begin:N (
\char_set_catcode_group_end:N )
\char_set_catcode_other:N \{
\char_set_catcode_other:N \}
\cs_new:Npn \BNVS_scan_Crl:nTTnFw #1 #2 #3 #4 #5 (
  \peek_regex_replace_once:NnTF \c__bnvs_Crl_open_regex ( ) (
    \BNVS_scan_close:nTTNnw ( #1 ) ( #2 ) ( #3 ) %{
    } ( #4 )
  ) (
    #5
  )
)
\group_end:
```

1.3.8 Raw units

GOBBLED

1.3.9 ⟨*csl(⟨item⟩)*⟩units

GOBBLED

1.3.10 ISR unit

GOBBLED

1.3.11 ISR unit

```
\regex_const:Nn \c__bnvs_IKS_regex {
```

1: the optional frame ⟨*id*⟩ with

2: the optional exclamation mark

```
  (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?
```

3: followed by a non empty short name.

```
  ( \ur{c__bnvs_S_regex} )
}
```

GOBBLED